

Task 4: Password Security & Authentication Analysis

Understand Password Storage (Hashing vs Encryption)

◆ Hashing

- One-way function (cannot be reversed back)
- Used for storing passwords securely
- Examples: MD5, SHA1, bcrypt

◆ Encryption

- Two-way (can decrypt back using key)
- Used for storing sensitive data like files/messages
- Example: AES

Identify hash types

MD5

- 32 hex characters
- Fast → weak → easily cracked

SHA1

- 40 hex characters
- Weak (collision attacks exist)

bcrypt

- Slow, salted
- Strong for password hashing

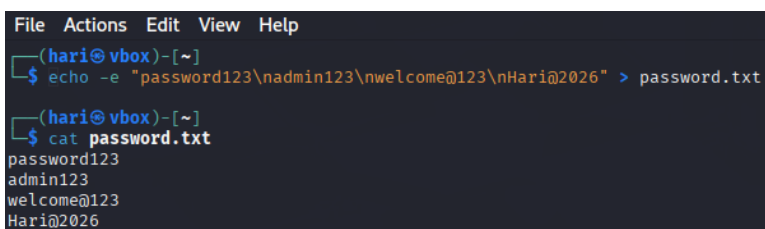
Generate password hashes

In Linux terminal:

Create a test password file:

```
echo -e "password123\nadmin123\nwelcome@123\nHari@2026" > passwords.txt
```

```
cat passwords.txt
```



```
File Actions Edit View Help
(hari@vbox)-[~]
$ echo -e "password123\nadmin123\nwelcome@123\nHari@2026" > password.txt
(hari@vbox)-[~]
$ cat password.txt
password123
admin123
welcome@123
Hari@2026
```

Generate MD5 hashes:

```
while read p; do echo -n "$p" | md5sum; done < passwords.txt > md5_hashes.txt
```

```
cat md5_hashes.txt
```

```
(hari@vbox)-[~]
$ while read p; do echo -n "$p" | md5sum; done < password.txt > md5_hashes.txt

(hari@vbox)-[~]
$ cat md5_hashes.txt
482c811da5d5b4bc6d497ffa98491e38 -
0192023a7bbd73250516f069df18b500 -
c7a85972e188c669f0d1e24528a772a3 -
f6f814471e89264206ac399b18ba4829 -
```

Generate SHA1 hashes:

```
while read p; do echo -n "$p" | sha1sum; done < passwords.txt > sha1_hashes.txt
```

```
cat sha1_hashes.txt
```

```
(hari@vbox)-[~]
$ while read p; do echo -n "$p" | sha1sum; done < password.txt > sha1_hashes.txt

(hari@vbox)-[~]
$ cat sha1_hashes.txt
cbfdac6008f9cab4083784cbd1874f76618d2a97 -
f865b53623b121fd34ee5426c792e5c33af8c227 -
5c9c83e88251dc90288910218600b691a446f31e -
d5153d948446590e1600bbc46ac52f6e2bceaa0f -
```

Understand brute force vs dictionary attack

Dictionary attack

- Tries common passwords from a wordlist
Example: rockyou.txt list

Brute force attack

- Tries all combinations (a-z, 0-9 etc.)
- Time-consuming but works if password is short

Crack weak hashes using John the Ripper

Install John

```
sudo apt update
```

```
sudo apt install john -y
```

```

(hari@vbox)-[~]
$ sudo apt install john -y
[sudo] password for hari:
Upgrading:
john john-data

Summary:
Upgrading: 2, Installing: 0, Removing: 0, Not Upgrading: 2030
Download size: 37.4 MB
Space needed: 582 kB / 11.1 GB available

Get:1 http://http.kali.org/kali kali-rolling/main amd64 john amd64 1.9.0-Jumbo-1+git20211102-0kali10 [14.7 MB]
2% [1 john 848 kB/14.7 MB 6%]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 john-data all 1.9.0-Jumbo-1+git20211102-0kali10 [22.8 MB]
Fetched 37.4 MB in 6min 49s (91.4 kB/s)
(Reading database ... 412030 files and directories currently installed.)
Preparing to unpack .../john.1.9.0-Jumbo-1+git20211102-0kali10_amd64.deb ...
Unpacking john (1.9.0-Jumbo-1+git20211102-0kali10) over (1.9.0-Jumbo-1+git20211102-0kali9) ...
Preparing to unpack .../john-data.1.9.0-Jumbo-1+git20211102-0kali10_all.deb ...
Unpacking john-data (1.9.0-Jumbo-1+git20211102-0kali10) over (1.9.0-Jumbo-1+git20211102-0kali9) ...
Setting up john-data (1.9.0-Jumbo-1+git20211102-0kali10) ...
Setting up john (1.9.0-Jumbo-1+git20211102-0kali10) ...
Processing triggers for wordlists (2023.2.0) ...
Processing triggers for kali-menu (2025.2.7) ...
Processing triggers for man-db (2.13.1-1) ...

```

Prepare file for John

MD5 hashes clean:

```

(hari@vbox)-[~]
$ cut -d " " -f1 md5_hashes.txt > md5_only.txt

(hari@vbox)-[~]
$ cat md5_only.txt
482c811da5d5b4bc6d497ffa98491e38
0192023a7bbd73250516f069df18b500
c7a85972e188c669f0d1e24528a772a3
f6f814471e89264206ac399b18ba4829

```

Run John (Dictionary attack):

```

(hari@vbox)-[~]
$ john --format=raw-md5 --wordlist=password.txt md5_only.txt
Using default input encoding: UTF-8
Loaded 4 password hashes with no different salts (Raw-MD5 [MD5 128/128 SSE2 4x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 4 candidates left, minimum 12 needed for performance.
password123      (?)
admin123         (?)
welcome@123      (?)
Hari@2026        (?)
4g 0:00:00:00 DONE (2026-01-23 09:49) 8.510g/s 8.510p/s 8.510c/s 34.04C/s password123..Hari@2026
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.

(hari@vbox)-[~]
$ john --show --format=raw-md5 md5_only.txt
?:password123
?:admin123
?:welcome@123
?:Hari@2026

4 password hashes cracked, 0 left

```

Crack SHA1 hashes using John:

```

(hari@vbox)-[~]
$ cut -d " " -f1 sha1_hashes.txt > sha1_only.txt

(hari@vbox)-[~]
$ cat sha1_only.txt
cbfdac6008f9cab4083784cbd1874f76618d2a97
f865b53623b121fd34ee5426c792e5c33af8c227
5c9c83e88251dc90288910218600b691a446f31e
d5153d948446590e1600bbc46ac52f6e2bceaa0f

(hari@vbox)-[~]
$ john --format=raw-sha1 --wordlist=password.txt sha1_only.txt
Using default input encoding: UTF-8
Loaded 4 password hashes with no different salts (Raw-SHA1 [SHA1 128/128 SSE2 4x])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
password123      (?)
admin123         (?)
welcome@123      (?)
Hari@2026        (?)
4g 0:00:00:00 DONE (2026-01-23 09:55) 26.66g/s 26.66p/s 26.66c/s 106.6C/s password123..Hari@2026
Use the "--show --format=Raw-SHA1" options to display all of the cracked passwords reliably
Session completed.

(hari@vbox)-[~]
$ john --show --format=raw-sha1 sha1_only.txt
?:password123
?:admin123
?:welcome@123
?:Hari@2026

4 password hashes cracked, 0 left

```

Why weak passwords fail (Analysis):-

- short (ex: 12345)
- common patterns (admin123, qwerty)
- reused passwords
- no symbols/uppercase
- predictable personal data (dob/name)

MFA (Multi-Factor Authentication):-

- Something you know → password/PIN
- Something you have → OTP/Authenticator app
- Something you are → fingerprint/faceID

Recommendations for Strong Authentication

- **Use strong password policy:**
 - **12–16+ characters**
 - **mix uppercase, lowercase, numbers, symbols**
 - **avoid common words**
 - **don't reuse passwords**
- **Use password managers**
- **Enable MFA everywhere**
- **Use bcrypt/argon2 hashing with salt**
- **Rate limiting / account lockout**
- **Monitor failed login attempts**